Replacement function

Document #:	P2826R0
Date:	2023-03-15
Project:	Programming Language C++
Audience:	EWG

Reply-to: Gašper Ažman <gasper.azman@gmail.com>

Contents

- 1 Introduction
- 2 Status
  - 2.1 Brief motivation and prior art
  - 2.2 Related Work
- 3 Proposal
  - 3.1 What if the constant expression evaluates to null?
- 4 Why this syntax
- 5 Nice-to-have properties
- 6 Use-cases
  - 6.1 deduce-to-type
  - 6.2 Traits-in-library for ad-hoc polymorphism and ABI stability
  - 6.3 expression-equivalent for `std::strong_order`
  - 6.4 interactions with `__builtin_calltarget()`
  - 6.5 interactions with templates
  - 6.6 interaction with “fixing surrogate deduction”
  - 6.7 programmable UFCS
  - 6.8 The “Rename Function” refactoring becomes possible in large codebases
  - 6.9 The “rename function” refactoring becomes ABI stable
  - 6.10 Changing the return type of a conversion function
  - 6.11 Witness tables for wrapper types (a-la Carbon)
- 7 References

§ 1 Introduction

This paper introduces a way to add a function to an overload set with a given signature.

Example:

```
struct S {};
struct U { operator S() const { return{}; } };

int g(S) { return 42; }
auto f(U) = &g;
long f(S) { return 1l; }
int h() {
  return f(U{}); // returns 42
}
```

§ 2 Status

This paper is in early stages of exploration. The R0 is released as early-feedback.

§ 2.1 Brief motivation and prior art

There are many cases in c++ where we want to add a function to an overload set without wrapping it.

A myriad of use-cases are in the standard itself, just search for *expression_equivalent* and you will find many.

The main thing we want to do in all those cases is:

— step 1: detect a call expression pattern

— step 2: call the appropriate function for that case

The problem is that the detection of the pattern and the appropriate function to call often don't fit into the standard overload resolution mechanism of C++, which necessitates a wrapper, such as a customization point object, or some kind of dispatch function.

This dispatch function, however it's employed, has no way to distinguish prvalues from other rvalues and therefore cannot possibly emulate the copy-elision aspects of *expression-equivalent*. It is also a major source of template bloat. The proposed mechanism also solves a number of currently unsolved problems in the language (see the use-cases section).

§ 2.2 Related Work

Roughly related were Parametric Expressions [P1221R1], but they didn't interact with overload sets very well.

This paper is strictly orthogonal, as it doesn't give you a way to rewire the arguments, just substitute the function that's actually invoked.

§ 3 Proposal

We propose a new kind of function definition of the form

function-body: … = *constant-expression* ;

where the *constant-expression* evaluates to a pointer-to-function or pointer-to-member-function.

Example (illustrative):

```
int takes_long(long i) { return i + 1; };
auto takes_int(int) = &takes_long;

void test1() {
  takes_int(1); // calls takes_long and implicitly converts 1 into a long.
}

template <typename T>
struct Container {
  auto cbegin() const -> const_iterator;
  auto begin() -> iterator;
  auto begin() const = &cbegin; // saves on templates
};
```

That's pretty much it - you already know how it works (TM).

However, let's spell the rules out, just for clarity:

We have two parts to any function definition: the declarator, and the body.

The function signature as declared in the declarator participates in overload resolution normally (the constant expression is not part of the immediate context).

If overload resolution picks this signature, then, *poof*, instead of substituting this signature into the call-expression, the replacement function is substituted. If this renders the program ill-formed, so be it.

§ 3.1 What if the constant expression evaluates to null?

That renders calling that overload ill-formed (and thus effectively means the same as a programmable `= delete`), unless the member function is declared `virtual`, in which case you can now spell conditionally-implemented overrides. Yay.

For virtual functions, though, the signature needs to match in the same way as signatures of overrides have to match the virtual interface declaration, otherwise we have a problem with ABI.

Example:

```
template <bool enable_implementations>
struct A {
  long g(int) { return 42; }
  long h(int) = enable_implementations ? &g : nullptr;
  virtual long f(int) = enable_implementations ? &g : nullptr;
};
struct Concrete : A<true> {};
struct Abstract : A<false> {};
struct Concrete2 : Abstract { long f(int) override { return 3; } };
void impl() {
  Concrete x;  // ok
  x.h(2);      // ok, 42

  Concrete2 z; // ok
  z.f(2);      // ok, 3
  z.h(2);      // Error, h is explicitly deleted

  Abstract y;  // Error, f(int) is abstract
};
```

§ 4 Why this syntax

— syntax noted actually already exist, just look for =delete

— it works for `0` as “not-provided” and =delete as no body, consistently

§ 5 Nice-to-have properties

— removal of library-defined dispatch from callstacks

— optimizers have to work way less hard

— major symbol reduction

— no issues with expressions - we don’t modify expressions (compare: parametric expressions)

— No extra work for overload resolution - this feature has no bearing on overload resolution, as the *`constant-expression`* is evaluated at declaration time.

§ 6 Use-cases

§ 6.1 deduce-to-type

The problem of template-bloat when all we wanted to do was deduce the type qualifiers has plagued C++ since move-semantics were introduced, but gets much, much worse with the introduction of (Deducing This) [P0847R7] into the language.

This topic is explored in Barry Revzin’s [P2481R1].

This paper provides a way to spell this, although the spelling is not ideal, and a specific facility for this functionality might be desirable as a part of a different paper.

Observe:

```
struct A {};
struct B : A {};

template <typename T>
auto _f_impl(T&& x) {
  // only ever gets instantiated for A&, A&&, A const&, and A const&&
};

// TODO define cvref-derived-from
template <typename D, typename B>
concept derived_from_xcv = std::derived_from<std::remove_cvref_t<D>, B>;

template <std::derived_from<A> T>
void f(T&&) = &_f_impl<copy_cvref_t<T, A>>;

void use_free() {
  B b;
  f(b);                // OK, calls _f_impl(A&)
  f(std::move(b));     // OK, calls _f_impl(A&&)
  f(std::as_const(b)); // OK, calls _f_impl(A const&)
}
```

Or, for an explicit-object member function:

```
struct C {
  void f(this std::same_as<C> auto&& x) {} // implementation
  template <typename T>
  void f(this T&& x) = static_cast<void (*)(copy_cvref_t<T, C>&&)>(f);
   // or, with __builtin_calltarget (P2825) - helps if you just want to
        write the equivalent expression
  template <typename T>
   void f(this T&& x) = __builtin_calltarget(std::declval<copy_cvref_t<T,
        C>&&>().f());
};
struct D : C {};

void use_member() {
  D d;
  d.f();               // OK, calls C::f(C&)
  std::move(d).f();    // OK, calls C::f(C&&)
  std::as_const(d).f(); // OK, calls C::f(C const&)
}
```

6.1.1 Don’t lambdas make this shorter?

Observe that we could also have just defined the inline lambda as the *constant-expression*:

```
struct A {};
struct B : A {};
```

```
// saves on typing?
template <std::derived_from<A> T>
void f(T&&) = +[](copy_cvref_t<T, A>&&) {};

void use_free() {
  B b;
  f(b);                // OK, calls f-lambda(A&)
  f(std::move(b));     // OK, calls f-lambda(A&&)
  f(std::as_const(b)); // OK, calls f-lambda(A const&)
}
```

While at first glance this saves on template instantiations, it's not really true (but it might save on executable size if the compiler employs COMDAT folding).

The reason is that the lambdas still depend on `T`, since one can use it in the lambda body.

This is very useful if we need to manufacture overload sets of *functions with identical signatures*, such as needed for type-erasure.

Continued example:

```
template <typename T>
int qsort_compare(T const*, T const*) = +[](void const* x, void const* y)
        {
    return  static_cast<int>(static_cast<T  const*>(x)  <=>  static_cast<T
        const*>(y));
  //                                  ^^                          ^^
};


// special-case c-strings
int  qsort_compare(char  const*  const*,  char  const*  const*)  =  +[](void
        const* x, void const* y) {
  auto const x_unerased = static_cast<char const* const*>(x);
  auto const y_unerased = static_cast<char const* const*>(y);
  return strcmp(*x_unerased, *y_unerased);
};

void use_qsort_compare() {
  std::vector<int> xs{1, 4, 3, 5, 2};
  ::qsort(xs.data(), xs.size(), sizeof(int),
                    __bultin_calltarget(qsort_compare(std::declval<int*>(),
        std::declval<int*>()));
             std::vector<char       const*>       ys{"abcd",       "abdc",
        "supercalifragilisticexpielidocious"};
  ::qsort(ys.data(), ys.size(), sizeof(char const*),
          __bultin_calltarget(qsort_compare("", "")));
}
```

So, while lambdas don't turn out to be a useful "template-bloat-reduction" strategy, they *do* turn out to be a very useful strategy for type-erasure.

§ 6.2 Traits-in-library for ad-hoc polymorphism and ABI stability

Consider a library that deals with matrices of generic types and knows broadcasting. The library is compiled separately and loaded dynamically, so it needs a stable ABI.

The library defines ABI-trait structs to be able to easily call into pre-compiled functions with hooks, such as the following “comparable” trait:

```
namespace lib {
  struct _ComparableTrait {
     using compare_function_t = std::strong_ordering (*)(void const*, void
         const*);
    compare_function_t compare;
  };
  template <typename A, typename B>
  constexpr _ComparableTrait Comparable = {
    .compare = +[](void const* px, void const* py) static noexcept
                        -> std::strong_ordering {
                    auto const& x = *static_cast<A const*>(px);
                    auto const& y = *static_cast<B const*>(py);
                    return x <=> y;
                };
  };
}
```

The library also defines an ABI stable, but private entry-point for a broadcast matrix compare:

```
struct MatrixRef {
    /* element access, size, element size and pointer to data, but not
        element type */
};
template <typename Element>
struct Matrix {
  /*...*/
  operator MatrixRef () const { }
};
/* broadcast cmp.compare() over matrix elements and return result,
        possibly in parallel */
auto _broadcast_impl(MatrixRef x, MatrixRef const& y, _ComparableTrait
        cmp) -> Matrix<std::strong_ordering>;

template <typename E1, typename E2>
auto   _broadcast_impl_helper(Matrix<E1>   const&,   Matrix<E2>   const&,
        _ComparableTrait = Comparable<E1, E2>) = &_broadcast_impl;

template <typename E1, typename E2>
auto   operator<=>(Matrix<E1>   const&   x,   Matrix<E2>   const&   y)   =
        __builtin_calltarget(_broadcast_impl_helper(x, y));
```

§ 6.3 expression-equivalent for `std::strong_order`

TODO

§ 6.4 interactions with `__builtin_calltarget()`

TODO

§ 6.5 interactions with templates

TODO

§ 6.6 interaction with “fixing surrogate deduction”

TODO

§ 6.7 programmable UFCS

TODO: if we get “pmfs are callable” one could substitute with a pointer-to-member-function and things would work even for free functions.

§ 6.8 The “Rename Function” refactoring becomes possible in large codebases

— we can finally rename a function (as opposed to the whole overload set) and not break things

See talk by Titus Winters (TODO find talk reference).

§ 6.9 The “rename function” refactoring becomes ABI stable

— we can finally move overload sets around and not break ABI in some cases since we basically gain function aliases.

§ 6.10 Changing the return type of a conversion function

— make example for changing return type of conversion function

§ 6.11 Witness tables for wrapper types (a-la Carbon)

— since you can do argument replacement, you can deduce-to-trait and generate witness tables

§ 7 References

[P0847R7] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-07-14. Deducing this.
https://wg21.link/p0847r7

[P1221R1] Jason Rice. 2018-10-03. Parametric Expressions.
https://wg21.link/p1221r1

[P2481R1] Barry Revzin. 2022-07-15. Forwarding reference to specific type/template.
https://wg21.link/p2481r1